EXPENSE & BUDGET TRACKER

A Microservices-Based Full-Stack Application

Submitted By	**Adhiraj Sharma**
Technology	Spring Boot 3 · React · Spring Cloud
Submitted On	November 11, 2026

1. Abstract

The Expense & Budget Tracker is a full-stack web application built using a microservices architecture with Spring Boot 3 on the backend and React on the frontend. The system enables users to register securely, log income and expense transactions in Indian Rupees (■), set monthly category-wise budgets with real-time overspending alerts, and view analytical dashboards including monthly trends, category comparisons, 3-month spending predictions, and yearly savings reports. JWT-based stateless authentication secures every API call. The application demonstrates industry-standard patterns: service discovery (Eureka), API gateway routing, inter-service communication via OpenFeign, and clean separation of concerns across five independently deployable microservices.

2. Objectives

- Implement secure user authentication using JWT and BCrypt password hashing.
- Provide full CRUD operations for income and expense transactions with filtering and pagination.
- Automatically calculate balance = total income – total expenses at all times.
- Allow users to set monthly category budgets with ≥80% usage warnings and exceeded alerts in ■.
- Display an interactive dashboard with Recharts: monthly bar chart and category pie chart.
- Perform month-over-month spending comparisons per category.
- Predict next-month spending using a 3-month moving average algorithm.
- Generate a yearly report with income, expenses, net savings, and savings rate %.
- Demonstrate microservices architecture with Eureka, API Gateway, and OpenFeign.

3. Requirements

3.1 Functional Requirements

#	Requirement	Priority
FR1	User registration and login with JWT token	High
FR2	Add, edit, delete income/expense transactions	High
FR3	Real-time balance calculation	High
FR4	Monthly budget with ■ alerts (80% warning, exceeded)	High
FR5	Transaction search and filter (text, type, category, month, amount range)	High
FR6	Dashboard with bar and pie charts	Medium

#	Requirement	Priority
FR7	Month-over-month category comparison	Medium
FR8	3-month moving average spending prediction	Medium
FR9	Yearly income/expense/savings report	Medium

3.2 Non-Functional Requirements

#	Requirement	Value
NFR1	Response time for API calls	< 500ms
NFR2	Password security	BCrypt strength 10
NFR3	Session management	JWT, 24-hour expiry
NFR4	Service independence	Each service has its own DB
NFR5	Fault tolerance	OpenFeign fallback if service down
NFR6	Mobile responsiveness	Tailwind CSS responsive layout

4. Why Microservices?

A monolithic architecture places all features in a single deployable unit. While simpler for small projects, this creates problems as the application grows: a bug in the analytics module can crash the entire app; scaling requires running full copies even if only one feature is under load; and large codebases become difficult to understand and maintain.

Microservices decompose the application into small, independent services, each with a single responsibility and its own database. Key advantages for this project:

Benefit	How It Applies
Independent deployment	user-service can be updated without touching transaction-service
Technology flexibility	Each service can use a different DB or language in future
Fault isolation	If budget-analytics-service crashes, users can still log transactions
Scalability	transaction-service can be scaled to 3 instances independently
Team parallel development	One student can own each service independently

5. System Architecture

The system follows a layered microservices architecture with these components:

Service	Port	Role	Database
discovery-server	8761	Eureka registry — all services register here	—
api-gateway	8080	Single entry point, JWT validation, request routing	—

Service	Port	Role	Database
user-service	8081	Register, login, JWT issuance, user profiles	user_db (H2)
transaction-service	8082	Transaction CRUD, search/filter, balance calculation	transaction_db (H2)
budget-analytics-service	8083	Budgets with ■ alerts, comparison, prediction, report	budget_db (H2)

Request Flow:

1. Browser sends request with JWT in Authorization header to API Gateway (port 8080). 2. Gateway's JwtAuthenticationFilter validates the token using the shared secret. 3. If valid, Gateway extracts userId and adds it as X-User-Id header. 4. Gateway routes request to the appropriate microservice by service name via Eureka. 5. The microservice processes the request, filters data by userId from X-User-Id. 6. budget-analytics-service calls transaction-service via OpenFeign (HTTP) to get spending data. 7. Response flows back through Gateway to the browser.

6. Database Design

6.1 user_db — users table

Column	Type	Description
id	BIGINT PK AUTO_INCREMENT	Unique user identifier
name	VARCHAR(100) NOT NULL	Full name of the user
email	VARCHAR(150) UNIQUE	Login email (must be unique)
password	VARCHAR(255) NOT NULL	BCrypt hashed password
created_at	TIMESTAMP	Account creation time

6.2 transaction_db — transactions table

Column	Type	Description
id	BIGINT PK AUTO_INCREMENT	Unique transaction ID
user_id	BIGINT NOT NULL	Owner (from X-User-Id header)
type	ENUM('INCOME','EXPENSE')	Transaction type
amount	DECIMAL(12,2) NOT NULL	Amount in ■ (e.g., 15000.00)
category	VARCHAR(50) NOT NULL	e.g., Salary, Housing, Food
description	VARCHAR(255)	Free-text note
date	DATE NOT NULL	Transaction date (YYYY-MM-DD)
created_at	TIMESTAMP	Record creation time

6.3 budget_db — budgets table

Column	Type	Description
id	BIGINT PK AUTO_INCREMENT	Unique budget ID
user_id	BIGINT NOT NULL	Owner
category	VARCHAR(50) NOT NULL	Budget category
budget_limit	DECIMAL(12,2) NOT NULL	Monthly spending limit in ■
month	INT NOT NULL	Month number (1–12)
year	INT NOT NULL	4-digit year
created_at	TIMESTAMP	Record creation time
UNIQUE	(user_id, category, month, year)	One budget per category per month

7. API Endpoints

All endpoints are accessed through the API Gateway at http://localhost:8080.

7.1 Authentication (public — no token required)

Method	Endpoint	Description
POST	/api/auth/register	Register new user → returns JWT token
POST	/api/auth/login	Login → returns JWT token
GET	/api/auth/me	Get current user profile (protected)

7.2 Transactions (protected — Bearer token required)

Method	Endpoint	Description
GET	/api/transactions	List transactions with filters: search, type, category, month, minAmount, maxAmo
POST	/api/transactions	Create new transaction
PUT	/api/transactions/{id}	Update transaction by ID
DELETE	/api/transactions/{id}	Delete transaction by ID
GET	/api/transactions/summary	Get total income, expense, balance

7.3 Budgets (protected)

Method	Endpoint	Description
GET	/api/budgets	List budgets for ?month&year with spent, percentUsed, status, message
POST	/api/budgets	Create monthly budget (category, budgetLimit, month, year)
PUT	/api/budgets/{id}	Update budget limit
DELETE	/api/budgets/{id}	Delete budget

7.4 Analytics (protected)

Method	Endpoint	Description
GET	/api/analytics/summary	Overall income, expense, balance
GET	/api/analytics/monthly	Monthly income vs expense (last N months)
GET	/api/analytics/categories	Spending by category
GET	/api/analytics/compare	Month-over-month category comparison
GET	/api/analytics/predict	3-month moving average prediction
GET	/api/analytics/report	Yearly report with savings rate

8. Security Design

8.1 Authentication Flow

The application uses stateless JWT (JSON Web Token) authentication. No session is stored on the server — the token itself carries the user identity.

Step	Action	Component
1	User submits email + password	React Login form
2	POST /api/auth/login reaches user-service	API Gateway (passes through — public route)
3	user-service verifies BCrypt password hash	UserServiceImpl.login()
4	JWT created with userId, email, 24h expiry	JwtTokenProvider.generateToken()
5	Token returned to browser	AuthController → LoginResponse
6	Browser stores token in localStorage	AuthContext.login()
7	Every API request adds 'Authorization: Bearer <token>'	Axios request interceptor
8	Gateway validates token signature + expiry	JwtAuthenticationFilter.filter()
9	Gateway adds X-User-Id header, routes request	API Gateway
10	Microservice reads X-User-Id, filters its data	TransactionController, BudgetController

8.2 Password Security

Passwords are hashed using BCrypt with strength factor 10 (≈100ms hash time). BCrypt is a one-way adaptive hashing function — the original password cannot be recovered from the hash. Each hash includes a unique salt, so identical passwords produce different hashes.

9. Testing

9.1 Unit Tests

Service	Test Class	Tests
user-service	UserServiceTest	register, login, duplicate email, bad password
transaction-service	TransactionServiceTest	create, update, delete, filter

Tests use JUnit 5 and Mockito. The Mockito mock-maker is configured to use 'mock-maker-subclass' (via mockito-extensions/org.mockito.plugins.MockMaker) for compatibility with JDK 17+.

9.2 Integration Testing (manual via Postman)

The Postman collection at postman/Expense_Tracker_API.postman_collection.json covers all 20+ API endpoints. The Login request automatically saves the JWT token to the {{token}} collection variable for use in all subsequent requests.

10. Screenshots

The following screenshots demonstrate the working application. All pages are mobile-responsive using Tailwind CSS.

Page	URL	Description
Login	http://localhost:5173/login	JWT login form with demo credentials hint
Register	http://localhost:5173/register	New user registration
Dashboard	http://localhost:5173/	Summary cards + bar chart + pie chart
Transactions	http://localhost:5173/transactions	Searchable, filterable transaction table
Add Transaction	http://localhost:5173/transactions/new	Form with type/category/amount/date
Budget	http://localhost:5173/budget	Progress bars with ■ alerts (green/orange/red)
Analytics	http://localhost:5173/analytics	Category comparison + prediction chart
Reports	http://localhost:5173/reports	Monthly breakdown table with savings rate
Eureka	http://localhost:8761	All 4 services registered (excluding discovery)
Swagger	http://localhost:8081/swagger-ui.html	Auto-generated API documentation

11. Limitations

- H2 in-memory databases: all data is lost when services restart. Switching to MySQL requires changing application.yml datasource settings.
- No email verification: users can register with any email address without confirmation.
- Single currency only: the app is hardcoded to INR (■). Multi-currency support would require exchange rate API integration.
- No role-based access control (RBAC): all authenticated users have the same permissions.

- Spending prediction accuracy: the 3-month moving average is a simple statistical method. ML-based models would give better accuracy.
- No real-time notifications: budget alerts are only shown when the user opens the Budget page.

12. Future Work

Feature	Description
MySQL persistence	Replace H2 with MySQL for durable data storage
Email alerts	Send email when budget is 80% used or exceeded
Receipt scanning	OCR to auto-extract transaction data from photos
Multi-currency	Support USD, EUR etc. with live exchange rates
Mobile app	React Native app consuming the same API Gateway
ML predictions	LSTM neural network for more accurate spending forecasts
Export to CSV/Excel	Download transaction history for offline analysis
Split expenses	Share expenses with family members (like Splitwise)
Bank account integration	Auto-import transactions via open banking APIs
CI/CD pipeline	GitHub Actions for automated build, test, and deploy

13. References

#	Reference
1	Spring Boot Documentation — https://docs.spring.io/spring-boot/docs/3.3.x/reference/html/
2	Spring Cloud Gateway — https://docs.spring.io/spring-cloud-gateway/docs/current/reference/html/
3	Netflix Eureka — https://spring.io/projects/spring-cloud-netflix
4	OpenFeign — https://spring.io/projects/spring-cloud-openfeign
5	JWT (io.jsonwebtoken) — https://github.com/jwtk/jjwt
6	BCrypt — Spring Security PasswordEncoder documentation
7	React Documentation — https://react.dev
8	Tailwind CSS — https://tailwindcss.com/docs
9	Recharts — https://recharts.org/en-US/api
10	Axios — https://axios-http.com/docs/intro
11	React Router v7 — https://reactrouter.com/en/main
12	springdoc-openapi — https://springdoc.org/
13	JUnit 5 — https://junit.org/junit5/docs/current/user-guide/

#	Reference
14	Mockito — https://site.mockito.org/
15	Moving Average — https://en.wikipedia.org/wiki/Moving_average